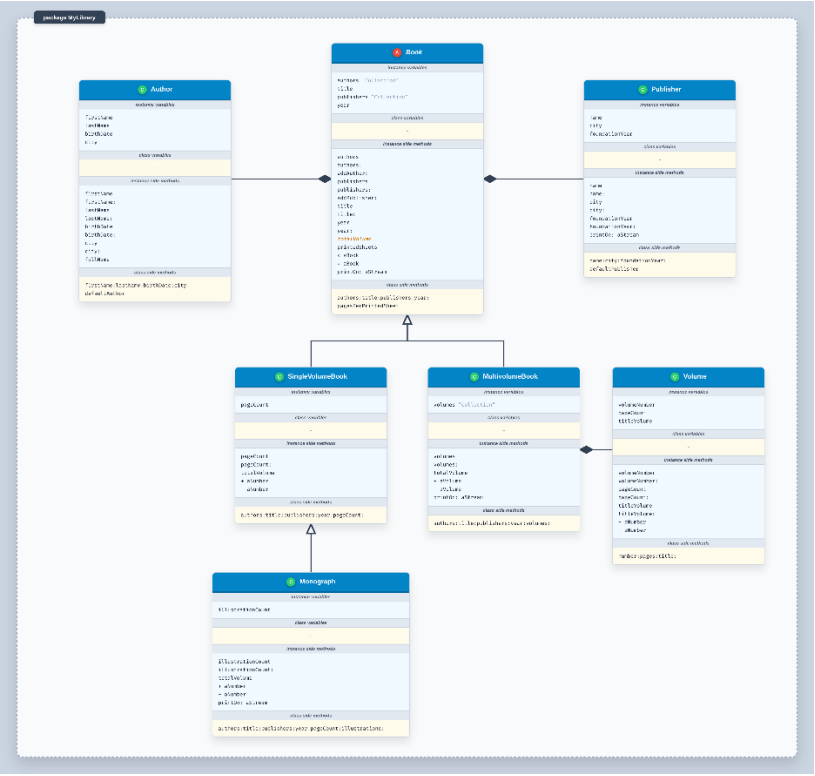
Першою складністю для мене стала правильна побудова самої ієрархії класів на етапі UML-моделювання. Я зразу відкинув ідею одного класу книга з купою необов'язкових полів та перевірок. Тому спочатку наша архітектура виглядала дуже спрощено і складалася лише з чотирьох класів: Book, SingleVolumeBook, Monograph та MultivolumeBook. Я планував, що ми будемо зберігати інформацію про авторів та видавництва як звичайні текстові рядки всередині базового класу Book, а багатотомник мав просто містити масив тексту або загальну суму сторінок. Проте, пізніше порадившись з Іриною, я відкинув і цю ідею, і вирішив додати до ієрархії клас Volume.



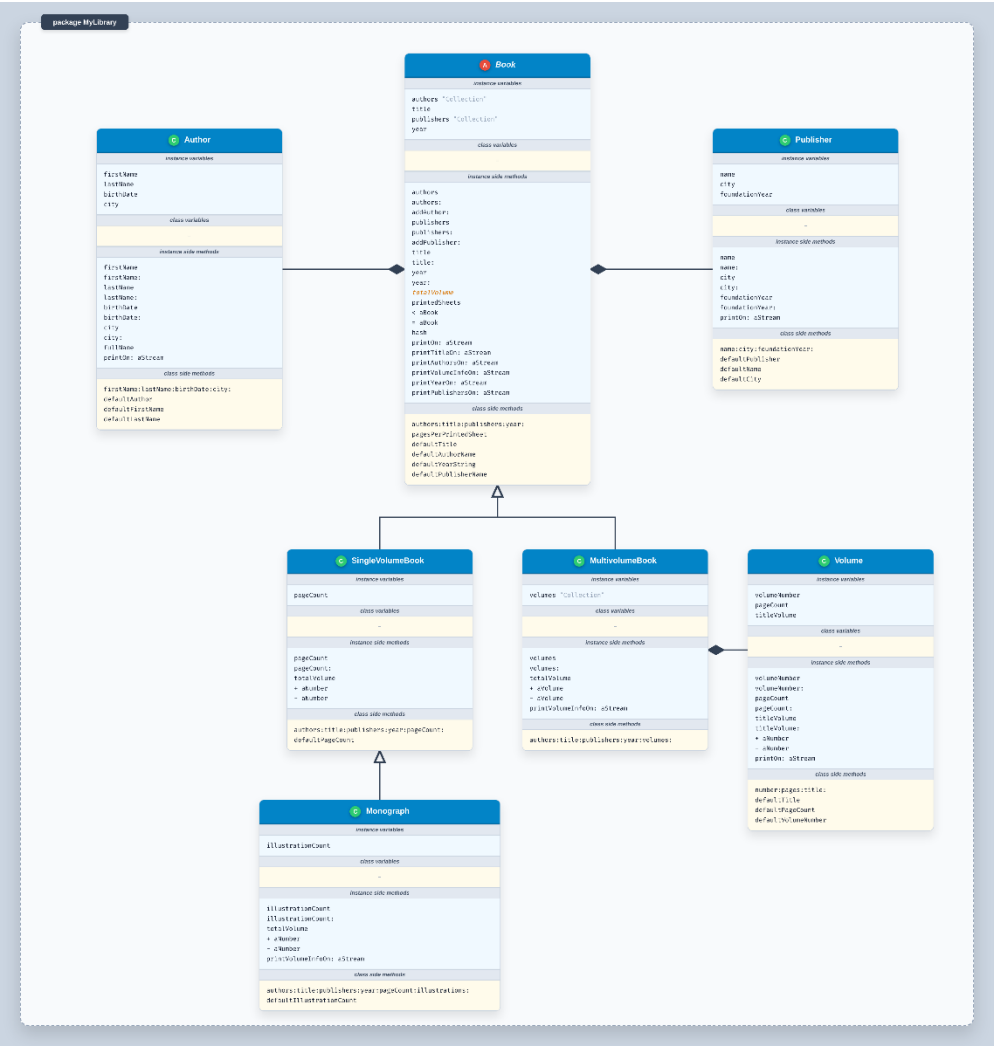
Далі під час аналізу ієрархії, я зрозумів, що Автор – це не рядок тексту з ім'ям, а його можна також виділити у окрему сутність з власною датою народження та містом. Видавництво – це окрема організація з роком заснування, назвою та містом. Тому я розширив ієрархію, додавши повноцінні класи-композити Author та Publisher. Завдяки цьому базовий клас Book почав працювати з

колекціями справжніх об’єктів, що, на мою думку, зробило ієрархію більш гнучкою.



Базовий клас Book я зробив абстрактним. Він взяв на себе відповідальність за спільні атрибути (назва, рік, колекції авторів та видавництв), але свідомо відмовився знати, як рахувати загальний обсяг сторінок, делегувавши це абстрактному методу totalVolume. Тобто нащадки (SingleVolumeBook, MultivolumeBook, Monograph) самостійно вирішують, як обчислювати свій обсяг.

На етапі написання методів я зіткнувся з проблемою значень за замовчуванням та проблемою примітивних типів даних для дат. Спочатку була спокуса передавати роки народження чи видання як звичайні цілі числа або рядки. Проте я вирішив все ж, що буду використовувати для цього Date та Year. Для цього потрібно було написати додаткову логіку валідації у сетерах, але, на мою думку, це зробило систему надійнішою.



Доволі несподіваною для мене виявилась проблема чистоти коду, на яку вказав аналізатор Pharo. Коли я писав перший варіант методу виведення printOn: в класі Book, то метод виявився занадто великим, з великою кількістю умов перевірки на порожні поля та ітерацій по колекціях, тому аналізатор дав на цей метод попередження high cyclomatic complexity. Тому я розділив один великий метод на п’ять маленьких, кожен з яких відповідає лише за свою частину тексту (назву, авторів, обсяг тощо). Крім того, я виніс значення за замовчуванням у константи на рівні класу, щоб можна було по-різному створювати екземпляри.

А далі я писав тести до створених мною класів Author, Publisher та Book та виправляв знайдені помилки у методах класів.